

JobPortal.AI — Distributed Microservices Recruitment Platform

An Enterprise Distributed Recruitment Engine featuring 10+ Spring Boot Microservices, Google OAuth2 SSO, Gmail SMTP 6-Digit OTP Email Verification, AI Skill Compatibility Matching Engine, Dynamic Drag-and-Drop Resume Builder, and Database-per-Service Architecture.

Project Title	JobPortal.AI (ZoshHire Platform)	Domain	Distributed Systems & AI Recruitment
Architecture Pattern	Microservices (Spring Cloud Framework)	Frontend Framework	React 18, Vite, Redux Toolkit, Tailwind
Backend Framework	Java 17 / Spring Boot 3.4+, Spring Cloud	Persistence Layer	PostgreSQL 16 (6 Dedicated DB Instances)
Service Registry	Netflix Eureka Registry (Port 8761)	API Gateway	Spring Cloud Gateway MVC (Port 9000)
Configuration	Spring Cloud Config Server (Port 8888)	Event Streaming	Apache Kafka 7.6.1 & Zookeeper 7.6.1
Containerization	Docker Desktop, Docker Compose, Jib Plugin	Security Models	Google OAuth2, JWT, Gmail SMTP OTP, RBAC

Presentation & Synopsis Agenda Index

This project synopsis is organized according to the official presentation agenda criteria:

#	Agenda Section Title	Core Technical Content Covered	Page Target
01	1. Problem Statement	Monolithic limitations, database lock contention, single points of failure, bot signup abuse, screening overhead.	Page 2
02	2. Objectives	Microservices decomposition, Database-per-Service, OAuth2+OTP flow, AI match engine, Kafka pipeline.	Page 3
03	3. Technology Stack	Full stack framework tables, libraries, infrastructure baseline, and 4-member development team role division.	Page 4–5
04	4. Literature Survey	Historical recruitment evolution (Gen 1-3), academic references, Literature Comparison Matrix vs Indeed/LinkedIn.	Page 6
05	5. System Workflow / Architecture	Architecture Topology Diagram, Gateway routes, Database Schemas (6 DBs), OAuth2+OTP Sequence, React UI routes.	Page 7–12
06	6. Expected Outcome	Tangible deliverables, SLA performance benchmarks, societal impact, carbon elasticity, future roadmap.	Page 13–14
07	7. References	Formal academic papers, specifications (OAuth2 RFC 6749, JWT RFC 7519), software manuals, sign-off block.	Page 15

1. Problem Statement

1.1 Background Context & Industry Challenges

Online job portals serve as the digital backbone of contemporary talent acquisition. Millions of job seekers upload resumes and apply for positions across global portals daily. However, conventional online recruitment systems are engineered predominantly around monolithic software architectures. In a monolithic architecture, authentication, job listings, application processing, messaging, and search indexing reside inside a single code artifact and a shared, monolithic database instance.

During high-volume hiring drives or peak traffic bursts, these monolithic systems experience severe infrastructure degradation. Heavy read/write operations exhaust database connection pools, lock shared table rows, and cause cascading thread starvation across the entire platform.

1.2 Detailed Problem Identification

Through extensive code and structural analysis, seven primary problem domains were identified in existing portal systems:

1. **Monolithic Relational Database Lock Contention:** Concurrent transactions across job posts, candidate application submissions, and search indexing contend for identical PostgreSQL table locks, producing frequent deadlocks, elevated latency, and dropped connections.
2. **Single Point of Failure (SPOF):** An unhandled exception or memory leak in a secondary component (e.g., mail dispatch or PDF rendering) crashes the entire monolithic process, blocking core authentication and application pathways.
3. **Unverified Bot Registrations & Spam Signups:** Basic single-factor registration allows automated bots to create mass spam profiles, polluting employer applicant tracking pipelines and degrading platform trust.
4. **High Manual Resume Screening Fatigue:** Recruiter teams waste over 70% of initial sourcing cycles manually parsing non-standardized resume formats, introducing human fatigue and hiring delays.
5. **Inflexible Scaling & Wasteful Infrastructure Spend:** Monolithic systems force administrators to scale the entire monolithic application image even if only job search indexing experiences high traffic, multiplying cloud hosting expenses.
6. **Synchronous Mail Dispatch Latency:** Triggering transactional emails synchronously inside request-response execution loops adds 2 to 5 seconds of HTTP latency per application submission.
7. **Static Resume Management:** Job seekers are forced to create resumes in external tools and continuously upload static files, causing parsing errors and formatting distortions.

1.3 Problem Domain Impact Summary

Problem Dimension	Monolithic Legacy Systems	Operational Impact
Database Architecture	Single Shared Monolithic DB	High table locks, database connection starvation, deadlocks during peak loads.
System Reliability	Coupled Application Monolith	Single component error causes total platform outage (SPOF).
User Verification	Unverified Single-Factor Auth	Bot spam profiles, fake accounts, recruiter inbox pollution.
Candidate Screening	Manual Keyword Inspection	Screening delays, recruiter fatigue, missed qualified talent.
Infrastructure Cost	All-or-Nothing Monolithic Scale	Inefficient cloud resource utilization, inflated hosting costs.

2. Objectives

2.1 Primary Project Goal

The primary objective of **JobPortal.AI** is to architect, implement, and validate a highly scalable, fault-tolerant, distributed microservices recruitment platform. By decoupling core domains into 10+ autonomous microservices and enforcing isolated database per service persistence, the platform guarantees high availability, sub-100ms API response times, and intelligent AI candidate screening.

2.2 Key Technical Objectives

To solve the identified problem statements, JobPortal.AI establishes seven core technical objectives:

- 1. Architect Domain-Driven Microservices Stack:** Deconstruct recruitment workflows into independent Spring Boot microservices: API Gateway, User Service, Job Service, Company Service, Application Service, Resume Service, Preference Service, AI Service, Notification Service, Eureka Discovery Server, and Config Server.
- 2. Implement Database-per-Service Isolation Pattern:** Provision 6 isolated PostgreSQL 16 database containers (`userdb:5433`, `companydb:5434`, `jobdb:5435`, `applicationdb:5436`, `preferencedb:5439`, `resumedb:5440`) to completely eliminate cross-domain database lock contention.
- 3. Engineer High-Security Hybrid OAuth2 + Mandatory OTP Email Verification:** Integrate Google OAuth2 Single Sign-On with a mandatory Gmail SMTP 6-digit OTP verification pipeline, assigning `INACTIVE` status until the code is validated.
- 4. Deliver AI Candidate Matching & Screening Engine:** Implement an AI service that calculates real-time candidate skill compatibility match percentages (\$MatchScore\$) against job requirement criteria.
- 5. Provide Live Interactive Drag-and-Drop Resume Builder:** Develop a dynamic React resume builder with instant section updates and dynamic PDF export capability.
- 6. Construct Asynchronous Event-Driven Messaging Pipeline:** Integrate Apache Kafka 7.6.1 and Zookeeper 7.6.1 for background notification queueing.
- 7. Automate Containerized Deployment Orchestration:** Package microservices into Docker images via Google Jib and manage multi-container stack orchestration using Docker Compose.

2.3 Functional Matrix & System Boundaries

Target User Role	In-Scope System Capabilities	Target Quality Benchmark
Job Seekers	Google OAuth2 / OTP login, profile management, real-time job search & filter, AI match evaluation, live drag & drop resume builder, application tracking.	Sub-100ms response time, mobile responsiveness.
Employers	Company profile setup, verification workflow, job creation & editing, applicant review pipeline, AI resume match rating, subscription management.	Zero cross-domain locks, automated candidate ranking.
System Admin	User account activation/suspension, employer verification, global platform metrics, job metadata administration.	Centralized audit & security administration.

3. Technology Stack

3.1 Comprehensive Framework & Component Matrix

JobPortal.AI utilizes a production-grade enterprise technology stack. Backend microservices are built on Java 17 and Spring Boot 3.4+, while the frontend is constructed using React 18, Vite, and Redux Toolkit.

Layer / Domain	Framework / Package	Version	Architectural Purpose
Language Platform	Java (JDK)	17 LTS	Core backend runtime environment.
Backend Core	Spring Boot	3.4.2	Microservice framework & REST controller engine.
Cloud Ecosystem	Spring Cloud	2023.0.0	API Gateway, Eureka Discovery, Config Server.
Security Chain	Spring Security / OAuth2	6.x	Google OAuth2 SSO, JWT parsing, Security Filter Chain.
Persistence Engine	PostgreSQL	16.0	Relational databases (6 containerized DBs).
ORM Layer	Spring Data JPA / Hibernate	7.2.1	Object-relational mapping & repositories.
Messaging Queue	Apache Kafka & Zookeeper	7.6.1	Asynchronous event streaming & notifications.
Frontend Core	React	18.3.1	Client UI rendering & component model.
Frontend Bundler	Vite	5.4.x	Fast HMR bundler and dev server.
State Management	Redux Toolkit	2.2.x	Global app state & async Thunks.
UI & Styling	Tailwind CSS & Shadcn UI	3.4.x	Utility-first design & UI primitives.
Form Validation	React Hook Form + Zod	3.x / 7.x	Client-side validation & schema enforcement.
Containerization	Docker & Docker Compose	24.0 / 2.20	Multi-container local stack orchestration.
Container Packaging	Google Jib Maven Plugin	3.5.1	Daemonless Java container packaging.
Build Automation	Apache Maven	3.9.x	Multi-module reactor build system.

3.2 4-Member Engineering Team Role Division & Ownership

To demonstrate clear project ownership, the engineering tasks across the codebase are structured among the 4 project team members as follows:

Team Member & Assigned Role	Primary Subsystems Handled	Core Technical Deliverables
Member 1: Distributed Backend & DevOps Lead Vikas Prajapati & Vineet	API Gateway (`9000`), Eureka (`8761`), Config Server (`8888`), Kafka (`29092`), Notification Service (`9005`).	Central routing, dynamic load balancing, Git config repo, Kafka producers/consumers, Docker Compose orchestration, Jib packaging.
Member 2: Security & Identity Engineer Shiv & Ajay	User Service (`9001`), Preference Service (`9008`), `common-lib`, `userdb:5433`.	Google OAuth2 SSO (`OAuth2SuccessHandler`), Gmail SMTP 6-digit OTP verification pipeline, 15-min Access & 7-day Refresh JWT tokens, RBAC.
Member 3: Business Domain & AI Engine Developer Vikas Prajapati & Vineet	Job Service (`9003`), Company Service (`9002`), Application Service (`9004`), Resume Service (`9009`), AI Service (`9010`).	Job creation/search, company verification, applicant tracking pipeline, AI skill match scoring engine (`\$MatchScore\$`), live PDF resume compiler.
Member 4: Frontend Engineer & UI/UX Lead Shiv & Ajay	React 18 App (`5173`), Redux Toolkit Store, React Router v6, Tailwind CSS, Shadcn UI.	All UI pages (`Register`, `Login`, `VerifyOtp`, `Jobs`, `Resumes`, `AIScreening`, `AdminDashboard`), Redux Thunks, route guards, Zod validation.

4. Literature Survey

4.1 Evolution of Online Recruitment Architectures

Online recruitment platforms have evolved through three generation phases over the past 25 years. **First-generation systems (1995–2005)** functioned as static job bulletin boards (e.g., early Monster.com), where employers posted basic text notices and candidates manually emailed static Word documents. **Second-generation systems (2005–2018)** introduced relational databases and keyword searching (e.g., early LinkedIn and Indeed), but remained constrained by monolithic database locks and manual screening. **Third-generation systems (2018–Present)** leverage cloud microservices, asynchronous message queues, and artificial intelligence to deliver elastic scalability and automated candidate matching.

Academic literature (Fowler & Lewis, 2014; Newman, 2021) proves that decomposing monolithic software into autonomous microservices eliminates single points of failure and prevents database lock exhaustion. Furthermore, research in automated recruitment analytics (Garg et al., 2022) demonstrates that AI match scoring algorithms reduce candidate screening cycles by up to 68% while significantly improving candidate-role compatibility.

4.2 Comparative Literature Matrix

A comparative matrix evaluating existing commercial platforms against JobPortal.AI across 9 key technical dimensions is detailed below:

Benchmark Dimension	Indeed	LinkedIn	Monster	JobPortal.AI (Proposed)
Architecture Pattern	Hybrid / Legacy Core	Distributed Microservices	Monolithic Legacy	Microservices (Spring Cloud Framework)
Database Model	Centralized Sharded	Distributed Graph & Document	Central Relational DB	Database-per-Service (PostgreSQL 16)
Authentication	Standard OAuth2	Custom SSO / OAuth2	Password / Basic OAuth	Google OAuth2 + Mandatory OTP
Account Verification	Email Click Link	Email / SMS Code	Basic Verification	Gmail SMTP 6-Digit OTP Verification
AI Match Engine	Keyword Search Based	ML Vector Embeddings	Keyword Filter	Integrated AI Skill Matcher (\$MatchScore\$)
Interactive Resume Builder	Static PDF Upload	Profile Export Only	Basic Parser	Live Drag & Drop Builder + PDF Export
Event Broker Queue	Internal Queues	Apache Kafka	RabbitMQ	Apache Kafka 7.6.1 + Zookeeper
Configuration Model	Proprietary Config	Dynamic Config Service	Static Files	Spring Cloud Config Server ('8888')

4.3 Gaps Identified in Existing Literature & Commercial Systems

- Black-Box Screening Algorithms:** Commercial portals rank candidates using non-transparent algorithms that provide zero feedback to candidates on missing skills.
- Weak Single-Factor Registration:** Most portals allow instant registration without immediate OTP verification, producing high volumes of spam accounts.
- Static Resume Dependency:** Job boards require uploading static Word/PDF files, causing parsing errors and formatting loss.

4. **Synchronous Notification Delays:** Sending confirmation emails synchronously inside API request loops adds multi-second latency for users.

5. System Workflow / Architecture (Part I)

5.1 Microservices Architecture Topology

JobPortal.AI is engineered following a clean microservices architecture topology. All incoming requests from client web browsers hit the **Spring Cloud API Gateway** (**port 9000**). The Gateway consults the **Netflix Eureka Service Registry** (**port 8761**) to resolve container IPs dynamically and load-balance traffic across microservices.

DISTRIBUTED MICROSERVICES ARCHITECTURE TOPOLOGY

```

+-----+
| REACT 18 FRONTEND CLIENT (Port 5173) |
+-----+
| HTTP / REST Calls
v
+-----+
| SPRING CLOUD API GATEWAY (Port 9000) |
| [Route Predicates | CORS Filter | JWT Header Forwarding Filter] |
+-----+
| | | |
v v v v
+-----+ +-----+ +-----+ +-----+
| USER SERVICE | | JOB SERVICE | | COMPANY SERV | | APP SERVICE |
| (Port 9001) | | (Port 9003) | | (Port 9002) | | (Port 9004) |
+-----+ +-----+ +-----+ +-----+
| | | |
v v v v
+-----+ +-----+ +-----+ +-----+
| PostgreSQL | | PostgreSQL | | PostgreSQL | | PostgreSQL |
| userdb:5433 | | jobdb:5435 | | companydb:5434 | | appdb:5436 |
+-----+ +-----+ +-----+ +-----+
| | | |
+-----+ +-----+ +-----+ +-----+
| Kafka Producer Events
v
+-----+
| APACHE KAFKA BROKER (Port 29092) |
| Zookeeper Coordination (Port 2181) |
+-----+
|
v Consumer Events
+-----+
| NOTIFICATION SERVICE (Port 9005) |
| Gmail SMTP Transactional Engine |
+-----+

```

5.2 Microservice Domain Catalog & Port Mappings

Microservice Name	Port	Database / Infrastructure	Functional Responsibility Scope
API Gateway	9000	Spring Cloud Gateway MVC	Central entry point, reverse proxy, CORS filter, JWT routing.
User Service	9001	PostgreSQL (`userdb:5433`)	User auth, Google OAuth2, OTP mail verification, JWT, RBAC.
Company Service	9002	PostgreSQL (`companydb:5434`)	Company profile registration, verification status, corporate assets.
Job Service	9003	PostgreSQL (`jobdb:5435`)	Job posting lifecycle, full-text search, filtering, bookmarking.
Application Service	9004	PostgreSQL (`applicationdb:5436`)	Job application pipeline, tracking status, screening workflow.
Notification Service	9005	Apache Kafka Consumer (`29092`)	Asynchronous event-driven emails via Gmail SMTP API.
Preference Service	9008	PostgreSQL (`preferencedb:5439`)	Candidate job preferences & alert recommendation criteria.
Resume Service	9009	PostgreSQL (`resumedb:5440`)	Live drag & drop resume builder state & dynamic PDF export.
AI Service	9010	AI Match Scoring Engine	AI resume-job skill compatibility scoring (\$MatchScore\$).
Config Server	8888	Git Repository (`config-repo`)	Centralized configuration management across all services.
Discovery Server	8761	Netflix Eureka Registry	Dynamic service registration, heartbeats, IP discovery.

5. System Workflow / Architecture (Part II)

5.3 Database Entity Models & Relational Schema Specifications

Each microservice manages its own isolated PostgreSQL relational schema. Below are the key data models defined across the platform databases:

1. User Entity Schema (`job\_portal\_user` on `userdb:5433`)

Column Name	Data Type	Constraints / Attributes	Description
`id`	BIGINT / Long	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier.
`email`	VARCHAR(255)	UNIQUE, NOT NULL	Primary authentication & contact email.
`password`	VARCHAR(255)	NULLABLE (for Google OAuth)	BCrypt encrypted password string.
`full_name`	VARCHAR(255)	NOT NULL	User's full name.
`role`	VARCHAR(50)	ENUM ('JOB_SEEKER', 'EMPLOYER', 'ADMIN')	Role-based authorization access level.
`auth_provider`	VARCHAR(50)	ENUM ('LOCAL', 'GOOGLE')	Authentication provider source.
`google_id`	VARCHAR(255)	NULLABLE	Google OAuth2 subject identifier.
`verified`	BOOLEAN	DEFAULT false	Account email OTP verification status.
`status`	VARCHAR(50)	ENUM ('ACTIVE', 'INACTIVE', 'SUSPENDED')	Current account lifecycle status.
`reset_token`	VARCHAR(255)	NULLABLE	Password reset verification string.

2. Job, Application, Company & Resume Schemas

Entity Model	Primary Columns	Foreign Keys / Domain References	Core Purpose
`Job`	id, title, description, location, job_type, salary_min, salary_max, status, deadline	company_id (References Company Service)	Stores active job posts & salary metadata.
`Application`	id, job_id, user_id, resume_id, status, cover_letter, match_score, applied_at	job_id, user_id, resume_id (Cross-service Keys)	Tracks applicant status & AI match rating.
`Company`	id, user_id, name, description, website, location, logo_url, verified	user_id (Employer Owner Key)	Stores corporate profiles & verification state.
`Resume`	id, user_id, title, summary, skills_json, experience_json, education_json	user_id (Candidate Owner Key)	Stores structured resume data for live editor.
`Otp`	id, email, code, expiry_time, used	email (User Reference Key)	Stores 6-digit email OTP verification codes.

5. System Workflow / Architecture (Part III)

5.4 Hybrid Google OAuth2 + Mandatory OTP Email Verification Flow

Authentication security is implemented via ``SecurityConfig.java``, ``OAuth2SuccessHandler.java``, and ``JwtProvider.java`` in the User Service (``port 9001``). Below is the complete step-by-step execution sequence of the newly engineered Google OAuth2 Single Sign-On and mandatory OTP verification flow:

Step	Component / Actor	Technical Execution Sequence
1	Client Browser	Candidate clicks 'Sign up with Google' on <code>`Register.jsx`</code> . Browser navigates to <code>`http://localhost:9000/oauth2/authorization/google`</code> .
2	API Gateway	Gateway forwards request to <code>`user-service`</code> (<code>`9001`</code>). Candidate authenticates on Google's consent page.
3	Google Auth Server	Google redirects back to callback endpoint: <code>`http://localhost:9000/login/oauth2/code/google?code=...`</code> .
4	<code>`OAuth2SuccessHandler`</code>	<code>`OAuth2Service.saveOrUpdate()`</code> extracts <code>`email`</code> , <code>`name`</code> , <code>`sub`</code> . If new user, creates entity with <code>`verified = false`</code> and <code>`status = INACTIVE`</code> .
5	<code>`MailServiceImpl`</code>	Service generates a 6-digit OTP code entity, saves to database, and mails it via Gmail SMTP (<code>`MimeMessageHelper`</code>).
6	Client Browser	Backend redirects browser to frontend OTP page: <code>`http://localhost:5173/verify-otp?email=user@gmail.com`</code> .
7	<code>`VerifyOtp.jsx`</code>	Candidate enters 6-digit code. React form issues POST request to <code>`http://localhost:9000/auth/verify-otp`</code> .
8	<code>`JwtProvider`</code>	Backend validates OTP, sets <code>`verified = true`</code> , <code>`status = ACTIVE`</code> , generates 15-min Access Token & 7-day Refresh Token, and returns JSON payload.
9	Redux Store / Client	Frontend saves <code>`accessToken`</code> in <code>`localStorage`</code> , dispatches <code>`fetchCurrentUser()`</code> , and navigates candidate to their dashboard.

5.5 AI Candidate Skill Match Calculation Model

The AI Service (``port 9010``) parses candidate resume skills (S_R) against job requirements (S_J) to compute a compatibility percentage score:

AI Skill Match Score Formula:

$$MatchScore = \left(0.60 \cdot \frac{|S_R \cap S_J|}{|S_J|} + 0.25 \cdot \min\left(1, \frac{Exp_R}{Exp_J}\right) + 0.15 \cdot Match_{Pref} \right) \times 100\%$$

Where $S_R \cap S_J$ represents overlapping technical skills, Exp represents years of experience, and $Match_{Pref}$ evaluates location/job-type preference.

5. System Workflow / Architecture (Part IV)

5.6 Frontend React Architecture & Route Hierarchy

The frontend application is built using **React 18** and **Vite**, delivering instant rendering and dynamic state management. Global application state is managed by **Redux Toolkit**, organizing API interactions into slice thunks (``userThunk.js``, ``jobSlice.js``, ``companySlice.js``, ``applicationSlice.js``).

Route Path	React Component	Guard / Access Level	Page Responsibility
<code>`/`</code>	<code>`LandingPage.jsx`</code>	Public	Hero banner, platform metrics, featured job listings.
<code>`/login`</code>	<code>`Login.jsx`</code>	Public / Auth Guard	Email/Password login & Google OAuth2 URL token parsing.
<code>`/register`</code>	<code>`Register.jsx`</code>	Public	User signup with Role selector (Candidate / Employer).
<code>`/verify-otp`</code>	<code>`VerifyOtp.jsx`</code>	Public	6-Digit OTP verification form with mail status alert.
<code>`/forgot-password`</code>	<code>`ForgotPassword.jsx`</code>	Public	Triggers password reset link dispatch via Gmail SMTP.
<code>`/jobs`</code>	<code>`Jobs.jsx`</code>	<code>`ROLE_JOB_SEEKER`</code>	Job search feed, filter sidebar (salary, experience, location).
<code>`/resumes`</code>	<code>`Resumes.jsx`</code>	<code>`ROLE_JOB_SEEKER`</code>	Interactive live drag-and-drop resume builder & PDF export.
<code>`/employer/dashboard`</code>	<code>`Dashboard.jsx`</code>	<code>`ROLE_EMPLOYER`</code>	Employer analytics, active job statistics, applicant counts.
<code>`/employer/ai-screening`</code>	<code>`AIScreening.jsx`</code>	<code>`ROLE_EMPLOYER`</code>	AI candidate ranking, resume match scoring, candidate filter.
<code>`/admin/dashboard`</code>	<code>`AdminDashboard.jsx`</code>	<code>`ROLE_ADMIN`</code>	Platform user activation, company verification, system metrics.

5.7 Form Validation & Schema Enforcement

Client forms utilize **React Hook Form** integrated with **Zod** schema validation (``authSchemas.js``). Forms validate email format, minimum 6-character password strength, matching passwords, and 6-digit numerical OTP limits prior to dispatching network requests.

6. Expected Outcome

6.1 Tangible Technical Deliverables & System Performance SLAs

The successful deployment of JobPortal.AI achieves key performance benchmarks and operational SLAs across the recruitment pipeline:

Operational Metric	Monolithic Baseline	JobPortal.AI Expected Outcome
API Gateway Response Time	450ms – 1200ms	Sub-100ms average latency via Spring Cloud Gateway routing.
Database Lock Contention	High table deadlocks under load	Zero cross-domain locks via 6 isolated PostgreSQL databases.
System Availability SLA	95.0% (SPOF outages)	99.9% Uptime via independent container failure isolation.
Candidate Screening Cycle	3 to 7 Days (Manual scan)	Real-Time (under 2 seconds) AI candidate match scoring.
Mail Dispatch Latency	2s to 5s Blocking HTTP delay	Asynchronous (0ms client blocking) via Kafka messaging.

6.2 Societal Relevance & Carbon Elasticity

JobPortal.AI impacts society by democratizing employment access, reducing hiring bias through objective AI skill matching, and eliminating paper waste through dynamic digital resume exports. Elastic microservices scaling enables cloud containers to scale down during low-traffic periods, conserving data center power.

6.3 Future Scaling Roadmap

- Kubernetes (K8s) & Helm Orchestration:** Migrating from Docker Compose to Kubernetes with Horizontal Pod Autoscalers (HPA).
- Elasticsearch & Kibana Job Search:** Implementing an Elasticsearch cluster for sub-millisecond full-text search.
- WebRTC Video Screening:** Integrating automated AI video screening and interview scheduling.

7. References

7.1 Academic Publications & Technical Specifications

1. Fowler, M., & Lewis, J. (2014). *Microservices: a definition of this new architectural term*. ThoughtWorks Insights.
2. Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.
3. Garg, L., et al. (2022). *Artificial Intelligence in Human Resource Management: Automated Candidate Screening Algorithms*. Journal of Systems Software, 184, 111-125.
4. Hardt, D. (2012). *The OAuth 2.0 Authorization Framework*. IETF RFC 6749.
5. Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. IETF RFC 7519.
6. Spring Cloud Engineering Team. (2024). *Spring Cloud Gateway & Netflix Eureka Reference Guide*. VMware Tanzu.
7. Apache Software Foundation. (2024). *Apache Kafka Distributed Event Streaming Documentation*.

Project Evaluation & Verification Panel	Status / Stamp
Student Developer Team: Vikas Prajapati, Vineet, Shiv, Ajay Project Evaluator: Department Project Examination Board Department: Computer Science & Engineering Institution: Faculty of Engineering & Technology	APPROVED FOR SYNOPSIS EVALUATION Date: September 24, 2026 Signature: _____